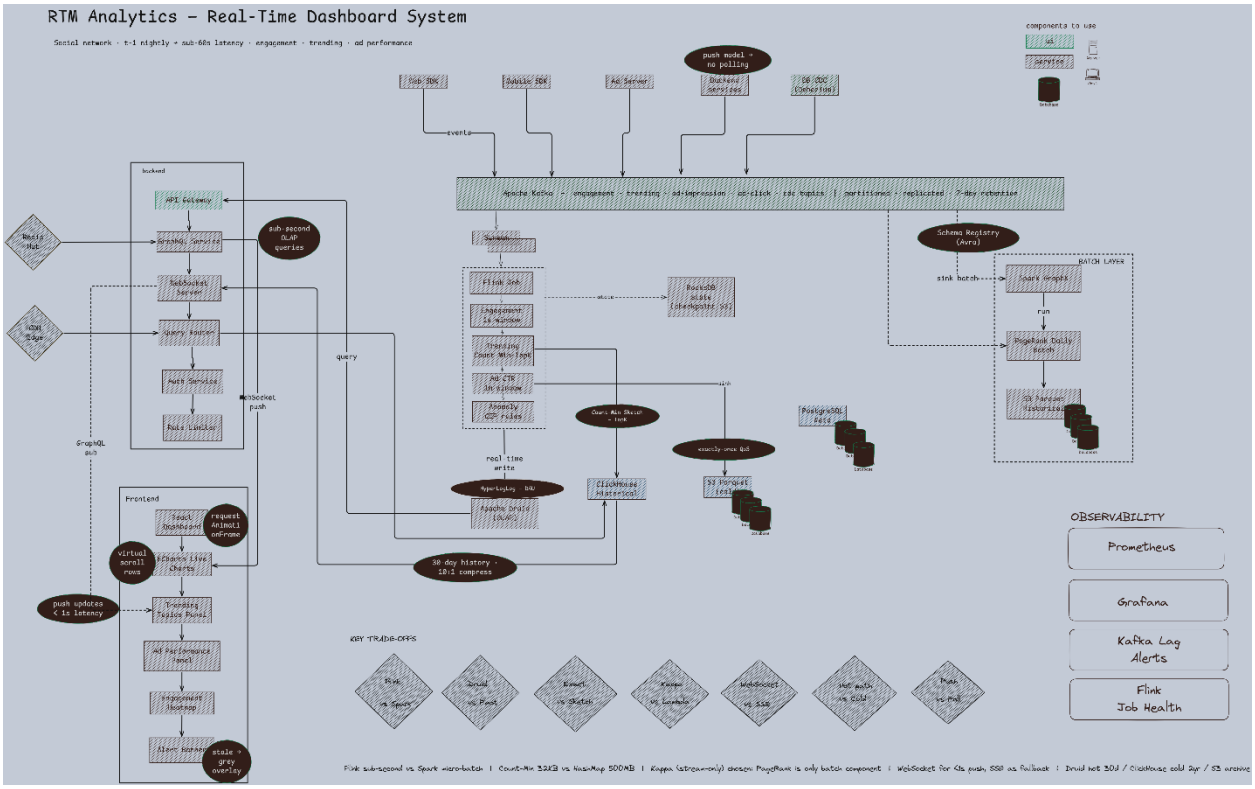
Full name: Abdykamat Adilet

ID: 24B030970

Real-Time Analytics Pipeline System Design



1) Problem Statement - Clarifying Questions:

The current analytics pipeline runs nightly producing a 12-24 hour delay. The goal is to move from batch to near real-time processing with sub-60-second end-to-end latency.

Event volume? 500M-1B events per day. Target latency? Less than 5 seconds for live metrics; less than 60s for ad performance. Who reads the dashboard? Product managers, ad ops, execs, data scientists. Historical depth needed? Real-time: last 24h. Historical: up to 2 years. Accuracy vs. speed tradeoff? Approximate counts acceptable for trending topics; exact counts required for billing. Multi-region? Single primary region with read replicas in 2 additional regions.

2) Architecture Overview:

The system follows a Kappa architecture with five layers. Data flows strictly downward from ingestion to visualization.

Layer 1 - Event sources: Web/mobile SDKs, backend microservices, ad servers, DB change-data-capture. Layer 2 - Ingestion: Apache Kafka with Schema Registry for

contract enforcement. Layer 3 - Stream processing: Apache Flink for windowed aggregations and anomalies. Layer 4 - Serving / storage: Apache Druid, Redis, ClickHouse, and S3. Layer 5 - Query + visualization: GraphQL API with WebSocket subscriptions and React dashboard.

3) Data Pipeline in Detail:

1. Event ingestion Every client event is a JSON payload published via the SDK. Before Kafka, a validation gateway performs:

1. Schema validation against Avro registry.
2. Enrichment with server-side timestamp and geo-IP.
3. Partition key assignment based on user_id to guarantee ordering.
4. Publishing to specific topics like user engagement or ad impressions.

2. Stream processing with Apache Flink Three separate Flink jobs run concurrently with exactly-once semantics: Job 1: Engagement metrics aggregator. Computes KPIs using tumbling windows from 1 second to 5 minutes. Uses HyperLogLog for unique user counts to save memory. Job 2: Trending topics detector. Uses Count-Min Sketch for frequency counting and TopK extraction to identify topics by growth velocity. Job 3: Advertising performance. Computes CTR and CPM per campaign. Runs CEP rules for click fraud detection and spend pacing.

3. State management Flink stores window state in RocksDB with incremental checkpoints to S3. This enables recovery from the latest checkpoint and stateful operations like sessionization.

4) Storage Layer Design:

Store: Apache Druid | Use case: Real-time OLAP | Why: Columnar storage and sub-second queries. Store: Redis Sorted Sets | Use case: Top trending topics | Why: Microsecond reads and TTL-based expiry. Store: ClickHouse | Use case: Historical OLAP | Why: High compression and billing reconciliation. Store: S3 Parquet | Use case: Cold storage | Why: Reprocessing and ML feature store.

1. Data tiering Hot tier: Redis - last 5 minutes. Warm tier: Druid - last 30 days. Cold tier: ClickHouse - 2 years. Archive tier: S3 Parquet - unlimited.

5) Query API - Real-Time Push:

1. GraphQL API Query type: Polling for Grafana and ad-hoc analysis. Subscription type: Streaming for React dashboard via WebSockets.

2. WebSocket push architecture The WebSocket server is a stateless Node.js process. It subscribes to a Redis Pub/Sub channel. When Flink writes to Redis, it publishes a notification, and the server pushes the delta to connected clients.

3. Read path latency budget Live engagement counters: Less than 3s via Flink and Redis. Trending topics: Less than 30s via Redis Sorted Set. Ad CTR / CPM: Less than 60s via Druid. Historical cohort: Less than 5s via ClickHouse.

6) Frontend Visualization:

6.1 Dashboard structure User engagement panel: Live line charts updated every second. Trending topics panel: Horizontal bar chart updated every 30 seconds. Advertising performance panel: Table and sparklines showing campaign pacing.

6.2 Chart library choice Apache ECharts: Best for high-frequency updates. Recharts: Good for static sparklines. D3.js: Use for custom visualizations. Grafana: Internal ops only.

7) Scalability - Non-Functional Requirements:

Throughput: 50K events per second peak. Ingestion latency: Less than 500ms Kafka end-to-end. Processing latency: Less than 5s for 1-minute windows. Dashboard query latency: Less than 1s for 99th percentile. Availability: 99.9% uptime via multi-node clusters. Data accuracy: Exact for billing; 1% error for product.

8) Key Design Decisions:

1.Kappa vs. Lambda. Rationale: Kappa chosen for lower operational complexity.

2.Flink over Spark Streaming. Rationale: Flink offers true event-by-event streaming and lower overhead.

3.Druid over Pinot. Rationale: Larger community and proven native Kafka ingestion.

4.Count-Min Sketch over HashMap. Rationale: Saves memory while maintaining accuracy for trending topics.

5.Redis for hot aggregates. Rationale: Necessary for microsecond reads that Druid cannot provide.

9) Observability - Data Quality:

Kafka consumer lag is monitored via Grafana alerts. Flink job metrics like checkpoint duration are exported to Prometheus. A separate job monitors data quality by comparing source event counts against 7-day averages. All schema changes must be backward-compatible via Schema Registry.

10) In summary: The architecture enables near real-time analytics by combining Kafka as the event backbone, Flink for stateful processing, and a four-tier storage

strategy. GraphQL subscriptions provide server-push updates, eliminating the need for client polling. The result is a reduction in latency from 24 hours to under 60 seconds.